Honors Numerical Analysis

MATH-UA 396

Yao Xiao

Spring 2023

Contents

0 Introduction **2**

1 IEEE Arithmetic **2**

1.1 Computer Representation of Numbers . . . 2

1.2 Rounding . . . 3

2 Solving Nonlinear Equations **3**

2.1 Bisection . . . 3

2.2 Secant Method . . . 4

2.3 Newton's Method . . . 4

3 Solving Nonlinear Systems **6**

3.1 Multivariable Vector-Valued Taylor Series . . . 8

3.2 Multivariate Newton's Method . . . 9

4 Optimization **9**

4.1 Single Variable Optimization . . . 9

4.2 Multivariate Optimization . . . 10

5 Numerical Linear Algebra **11**

5.1 Gaussian Elimination . . . 11

5.2 LU Factorization . . . 11

5.3 Cholesky Factorization . . . 12

5.4 Row Pivoting . . . 13

5.5 Conditioning and Backward Stability . . . 14

5.6 QR Factorization . . . 14

5.7 Gram-Schmidt (Triangular) Orthogonalization . . . 15

5.8 Householder Reflection (Orthogonal) Triangularization . . . 17

5.9 The Singular Value Decomposition . . . 19

1/23 Lecture

Instructor. Mike O'Neil, `oneil@cims.nyu.edu`.

Class time. Mondays & Wednesdays 2:00–3:15pm, 201 WWH.

Office hours. TBA, 1119 WWH.

Programming language. Julia, with `git` version control and Github Classroom.

Grading. Final 40%, Midterm 30%, Homework each 5%.

Course materials.

- Overton, *Numerical Computing with IEEE Floating Point Arithmetic*, SIAM, 2001.
- Driscoll and Braun, *Fundamentals of Numerical Computation*, SIAM, 2017.
- Trefethen and Bau, *Numerical Linear Algebra*, SIAM, 1997.
- Suli and Mayers, *An Introduction to Numerical Analysis*, Cambridge, 2003.

0 Introduction

- **Nonlinear Equations.** Nonlinear equations, nonlinear equations of several variables, and systems of nonlinear equations of several variables.
- **Linear Algebra.** Solving $A\vec{x} = \vec{b}$, eigenproblems $A\vec{v} = \lambda\vec{v}$, decompositions $A = QR = VSV^T$, and optimizations $y = \arg\min f(x)$.
- **Interpolation and Approximation.** Interpolants and approximation.
- **Numerical Integration.** Riemann integrals and quadratures.
- **Fast Algorithms.** Fast Fourier transform $O(n \log n)$ and Monte Carlos.

1/25 Lecture

1 IEEE Arithmetic

1.1 Computer Representation of Numbers

Fixed point and floating point representation. A fixed point representation specifies a fix number of bits for the integer and fraction part. For instance, with specified format "XXXX.XXXX", 7.5 becomes 0007.5000. A floating point representation, on the other hand, is represented $\pm M \times 2^E$, where $M \in [1, 2)$ is called the mantissa and E is called the exponent. For instance, $23 = (10111)_2 = (1.0111)_2 \times 2^4$, $0.125 = (0.001)_2 = (1.0)_2 \times 2^{-3}$, etc. A floating point number is one that can be stored *exactly* on a computer.

Single precision and double precision. A single precision number is 32 bits in total, which include 1 bit for the sign, 8 bits for the exponent, and 23 bits for the mantissa. For instance,

$$5.5 = 1.011 \times 2^2 = (\underbrace{0}_{\text{sign}} \,|\, \underbrace{E = 00000010}_{\text{exponent}} \,|\, \underbrace{M = 101100\cdots0}_{\text{mantissa, 23 bits}}). \tag{1}$$

On the other hand, a double precision number is 64 bits in total, which include 1 bit for the sign, 11 bits for the exponent, and 52 bits for the mantissa. Single precision and double precision are standardized by IEEE in the 1980s. In Julia, `x = 7.0` stores a `Float64` type variable, and `x = 7` stores a `Int64` type variable (an 8-byte integer).

To represent negative exponent E, the assumption is that E is offset by 1023, that is,

$$x = \pm M \times 2^{E-1023}. \tag{2}$$

In Julia, `bitstring(x)` returns the IEEE representation of `x`.

1.2 Rounding

Rounding. For instance, $(0.1)_{10} = (1.100\overline{1100})_2 \times 2^{-4}$, so the mantissa must be rounded and its approximation is stored as a floating point number. The dafault rounding strategy is rounding to the nearest. The important thing is the error made in rounding.

$$\text{Absolute rounding error} = |\text{round}(x) - x|, \quad \text{Relative rounding error} = \frac{|\text{round}(x) - x|}{|x|}. \tag{3}$$

IEEE standards. IEEE standard says that it must be the case that

$$\text{round}(a+b) = \text{round}(a) \underbrace{\bigoplus}_{\text{IEEE floating point addition}} \text{round}(b) = (a+b)(1+\delta), \tag{4}$$

where $|\delta| < \epsilon = \textsc{MachEp}$, where $\textsc{MachEp}$ denotes the machine precision. Machine precision is the distance between 1 and the next floating point number. In double precision, that is,

$$\epsilon = -(0|011\cdots1|00\cdots0) + (0|011\cdots1|00\cdots01) = 2^{-52}. \tag{5}$$

IEEE standard also says the following about relative accuracy of computations that

$$\frac{|\text{round}(a+b) - (a+b)|}{|a+b|} = |\delta| < \epsilon. \tag{6}$$

1/30 Lecture

2 Solving Nonlinear Equations

We can find solutions to linear equations such as $3x + 7 = 2$, but we cannot generally find solutions to nonlinear equations such as $\cos x + x^2 - 7 = 5$. This is a root finding problem. A *sufficient* condition for a solution $f(x) = 0$ (f is continuous) to exist is stated in the following theorem.

Theorem 2.1. If f is continuous on $[a, b]$, and if $f(a) \cdot f(b) \leq 0$, then there exists $x \in [a, b]$, such that $f(x) = 0$.

Proof. This is immediate from the Intermediate Value Theorem. □

2.1 Bisection

Idea. We will make use of Theorem 2.1. Split the interval in two, test the condition of a sign change and repeat.

Let $a_0 = a$, $b_0 = b$, and from the original interval, let $[a_l, b_l]$ be the interval obtained after l splittings. Then,

$$b_l - a_l = \frac{b_0 - a_0}{2^l} = \frac{L}{2^l}. \tag{7}$$

Let $x_l = \frac{a_l + b_l}{2}$ be the approximation of the solution to $f(x) = 0$ in the lth step. The question is, when do we stop the splittings, and how many steps of this algorithm do we take? If we want $|x_l - x^*| < \epsilon$ where x^* is the true solution $f(x^*) = 0$, then we choose l such that

$$|x_l - x^*| \leq \frac{b_l - a_l}{2} = \frac{L}{2^{l+1}} < \epsilon \implies 2^{l+1} > \frac{L}{\epsilon} \implies l > 1 + \log_2\left(\frac{L}{\epsilon}\right). \tag{8}$$

Convergence behavior. If $e_l = |x^l - x^*|$ represents the error in the lth step, then $e_{l+1} = \frac{1}{2}e_l$, meaning that the absolute error goes down by a factor of 2 in each iteration. Therefore, we can see that the bisection method is not efficient enough.

2.2 Secant Method

Idea. Bisection only used the sign information. If we use the values, we can do better. The idea is to approximate by a line with an initial guess, find the root of the line, update the guess and repeat. The secant line is

$$s(x) = \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x - x_0) + f(x_0), \tag{9}$$

which has the root

$$x = x_1 - f(x_1) \cdot \frac{x_1 - x_0}{f(x_1) - f(x_0)}. \tag{10}$$

We take this root as x_2 and repeat the process with x_0 substituted by x_1 and x_1 substituted by x_2. Let $f_k = f(x_k)$ for the sake of simplicity. The secant method generates a sequence of approximations to the root of f by

$$x_{k+1} = x_k - f_k \cdot \frac{x_k - x_{k-1}}{f_k - f_{k-1}}. \tag{11}$$

The convergence properties will be revisited later in this course.

2.3 Newton's Method

Idea. In addition to the values of the function, we can make use of both the value and the derivative of a function f at a single point to form an approximation line, find its root, and update and iterate just as in the secant method. The equation of the tangent line is

$$f(x) = f(x_0) + f'(x_0)(x - x_0), \tag{12}$$

which has the root

$$x = x_0 - \frac{f(x_0)}{f'(x_0)}. \tag{13}$$

We take this root as x_1 and repeat the process with x_0 substituted by x_1. The Newton's method generates a sequence of approximations to the root of f by

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \tag{14}$$

Note that (12) is the first-order Taylor approximation of f.

Convergence behavior. Let $e_k = |\xi - x_k|$ be the absolute error, where ξ is the true root of f. Newton's method converges quadratically, meaning that $e_{k+1} = Ae_k^2$ for some constant A. Note that the absolute error is very small, for instance, initially 10^{-1}. Then with only 5 iterations, the error will be near machine precision.

Theorem 2.2 (Theorem 1.8 from Suli and Mayers)**.** Suppose that f is twice continuously differentiable on the intercal $I_\delta = [\xi - \delta, \xi + \delta]$, $\delta > 0$, with $f(\xi) = 0$ and $f'(\xi) \neq 0$. Also, assume $A > 0$ such that

$$\left| \frac{f''(x)}{f'(y)} \right| \leq A, \qquad \forall x, y \in I_\delta. \tag{15}$$

If the initial guess satisfies $|\xi - x_0| \leq h$, with $h \leq \min\left(\delta, \frac{1}{A}\right)$, then the sequence $\{x_k\}$ defined by Newton's method (14) converges quadratically to ξ.

2/1 Lecture

Proof. By Taylor's Theorem, we have that

$$f(\xi) = 0 = f(x_0) + f'(x_0)(\xi - x_0) + \frac{f''(\xi_0)}{2}(\xi - x_0)^2, \tag{16}$$

where $\xi_0 \in [\xi, x_0] \subseteq I_\delta$. By Newton's method, we have that

$$\xi - x_1 = \xi - x_0 + \frac{f(x_0)}{f'(x_0)} = -\frac{f''(\xi_0)}{2f'(x_0)}(\xi - x_0)^2. \tag{17}$$

Therefore, we can deduce that

$$|\xi - x_1| \leq \frac{1}{2}\left|\frac{f''(\xi_0)}{f'(x_0)}\right| |\xi - x_0|^2 \leq \frac{1}{2}A\,|\xi - x_0|^2 \leq \frac{1}{2}Ah \cdot \frac{1}{A} = \frac{h}{2}. \tag{18}$$

Repeating this step k times, we can thus show that

$$|\xi - x_k| \leq \frac{h}{2^k} \implies \lim_{k\to\infty} x_k = \xi, \tag{19}$$

proving convergence. Furthermore, since

$$|\xi - x_{k+1}| \leq \frac{1}{2}\left|\frac{f''(\xi_k)}{f'(x_k)}\right| |\xi - x_k|^2, \tag{20}$$

we can see that

$$\lim_{k\to\infty} \frac{|\xi - x_{k+1}|}{|\xi - x_k|^2} = \lim_{k\to\infty}\left|\frac{f''(\xi_k)}{f'(x_k)}\right| = \frac{1}{2}\left|\frac{f''(\xi)}{f'(\xi)}\right| = \texttt{const}. \tag{21}$$

This is exactly the *definition of quadratic convergence.* □

Failures of Newton's method.

- When Newton's method fails to converge (quadratically), it is usually because $f'(\xi) = 0$ at the root. In this case, (15) may not hold, *i.e.*, the quantity may not remain bounded.
- For some initial guesses, Newton's method may fail to converge at all.

Rates of Convergence

For the convergent sequence $e_k = |x_k - x^*|$, we can consider the following limit

$$\lim_{k\to\infty} \frac{e^{k+1}}{e_k^\alpha} = \mu. \tag{22}$$

For instance, Bisection method has $\alpha = 1$ (*i.e.*, linear convergence), Secant method has approximately $\alpha = 1.62$, and Newton's method has $\alpha = 2$ (*i.e.*, quadratic convergence). For $\alpha = 1$, we further define the *order* of convergence as follows: For $\alpha = 1$ and $\mu < 1$, set $\rho = -\log_{10}\mu$ as the *asymptotic rate of convergence.* As an illustration, if $\mu = 1/10$, this means that

$$\frac{e_{k+1}}{e_k} \approx \frac{1}{10}, \tag{23}$$

i.e., the error goes down by 10% every iteration. That is to say that x_k gets one more correct digit every iteration, and we can see that

$$\rho = -\log_{10}\mu = -\log_{10}\frac{1}{10} = 1. \tag{24}$$

3 Solving Nonlinear Systems

A nonlinear system of equations in n variables $x_1, \cdots, x_n$ is given by

$$\begin{aligned} f_1(x_1, \cdots, x_n) &= 0, \\ &\vdots \\ f_n(x_1, \cdots, x_n) &= 0, \end{aligned} \tag{25}$$

which can be condense into the notation

$$\vec{f}(\vec{x}) = \vec{0}, \tag{26}$$

where

$$\vec{f} = \begin{pmatrix} f_1(\vec{x}) \\ \vdots \\ f_n(\vec{x}) \end{pmatrix}, \qquad \vec{x} = \begin{pmatrix} x_1 & \cdots & x_n \end{pmatrix}. \tag{27}$$

Note that:

- Bisection does not, in general, extend to higher dimensions, since sign changes do not necessarily indicate roots.
- The idea behind Secant method and Newton's method directly extends to higher dimensions: Linearize (approximate by a linear function) and find the root of the approximant.

Vector and Matrix Norms

Definition 3.1. Suppose $\vec{u}, \vec{v}, \vec{x} \in \mathbb{C}^n$, then $\|\cdot\|$ is a *norm* on $\mathbb{C}^n$ if

(1) $\|\vec{x}\| \geq 0$, and $\|\vec{x}\| = 0$ if and only if $\vec{x} = \vec{0}$.

(2) $\|\alpha\vec{x}\| = |\alpha|\|\vec{x}\|$ for $\alpha \in \mathbb{C}$.

(3) $\|\vec{u} + \vec{v}\| \leq \|\vec{u}\| + \|\vec{v}\|$ (triangle inequality).

The most useful norm in Numerical Analysis is the l_2-norm, defined as $\|\vec{u}\|_2 = \sqrt{\sum_i |u_i|^2}$. Other norms commonly used include:

- l_∞-norm: $\|\vec{u}\|_\infty = \max_i |u_i|$.
- l_1-norm: $\|\vec{u}\|_1 = \sum_i |u_i|$.
- l_p-norm: $\|\vec{u}\|_p = \left(\sum_i |u_i|^p\right)^{1/p}$.

Definition 3.2. Suppose $A, B \in \mathbb{C}^{m \times n}$, then $\|\cdot\|$ is a *matrix norm* if

(1) $\|A\| \geq 0$, and $\|A\| = 0$ if and only if $A = 0$.

(2) $\|\alpha A\| = |\alpha|\|A\|$ for $\alpha \in \mathbb{C}$.

(3) $\|A + B\| \leq \|A\| + \|B\|$.

If $\|\cdot\|$ is any vector norm, then the *induced matrix norm* is defined as

$$\|A\| = \max_{\|\vec{u}\|=1} \|A\vec{u}\| = \max_{\vec{u}} \frac{\|A\vec{u}\|}{\|\vec{u}\|}. \tag{28}$$

2/6 Lecture

For instance, if $\|\vec{u}\| = \|\vec{u}\|_1 = \sum_{i=1}^n |u_i|$, then the induced matrix norm on $A \in \mathbb{C}^{m\times n}$ is

$$\|A\|_1 = \max_{j=1,\cdots,n} \sum_{i=1}^n |A_{ij}|. \tag{29}$$

To see this, note that

$$\|A\vec{u}\|_1 = \left\| \sum_{j=1}^n u_j \vec{A}_j \right\|_1 \leq \sum_{j=1}^n |u_j| \|\vec{A}_j\|_1 \leq \left(\max_j \|\vec{A}_j\|_1 \right) \left(\sum_{j=1}^n |u_j| \right) = \left(\max_j \sum_{i=1}^n |A_{ij}| \right) \|\vec{u}\|_1. \tag{30}$$

Therefore, we have that

$$\frac{\|A\vec{u}\|_1}{\|\vec{u}\|_1} \leq \max_j \sum_{i=1}^n |A_{ij}|. \tag{31}$$

But taking $\vec{u}$ as a zero vector with only the jth entry as 1, we can see that $\|A\vec{u}\|_1 \geq \|\vec{A}_j\|_1$, so the equality must hold for some $\vec{u}$. Therefore, we can conclude that

$$\|A\|_1 = \max_{\vec{u}} \frac{\|A\vec{u}\|}{\|\vec{u}\|} = \max_{j=1,\cdots,n} \sum_{i=1}^n |A_{ij}|. \tag{32}$$

Theorem 3.3. Let $A \in \mathbb{R}^{m\times n}$ be a matrix. Then

$$\|A\|_2 = \sqrt{\max_j \lambda_j}, \tag{33}$$

where λ_j is the jth eigenvalue of $A^\top A$.

Proof. We have that

$$\|A\vec{u}\|^2 = (A\vec{u}, A\vec{u}) = (A\vec{u})^\top A\vec{u} = \vec{u}^\top A^\top A\vec{u}. \tag{34}$$

Note that $A^\top A$ is real and symmetric, it is diagonalizable, so we can write

$$\|A\vec{u}\|^2 = \vec{u}^\top P^\top D P\vec{u}, \tag{35}$$

where D is a diagonal matrix of eigenvalues and P is orthogonal (*i.e.*, $P^\top = P^{-1}$). Therefore, we can compute that

$$\max_{\|\vec{u}\|=1} \|A\vec{u}\|_2^2 = \max_{\|\vec{u}\|=1} \|\vec{u}^\top P^\top D P\vec{u}\|_2^2 = \max_{\|\vec{y}\|=1} \|\vec{y}^\top D\vec{y}\|_2^2 = \max_{\|\vec{y}\|=1} \sum_{i=1}^n y_i^2 \lambda_i = \max_i \lambda_i, \tag{36}$$

where we can change $\vec{y} = P\vec{u}$ without changing the norm since P is diagonal, and the proof of the last equality is left as an *exercise*. □

Condition number

The *condition number* of the problem is the sensitivity of the "problem" at the solution. For instance, consider a function $y = f(x)$, the how sensitive is y to x?

- In an absolute sense, we can write that

$$|y - y'| = C(x)|x - x'|, \tag{37}$$

  where $C(x)$ is the absolute condition number at x. Note that $C(x) \approx |f'(x)|$ when x' is close to x.

- In a relative sense, we can write that

$$\frac{|y' - y|}{|y|} = K(x)\frac{|x' - x|}{|x|}, \tag{38}$$

  where $K(x)$ is the relative condition number at x. Note that $K(x) \approx \left|\frac{xf'(x)}{f(x)}\right|$ when x' is close to x.

Conditioning for matrix equations. Consider solving $A\vec{x} = \vec{b}$, where the input is $\vec{b}$ and the output is $\vec{x} = A^{-1}\vec{b}$. Let $\|\vec{b}' - \vec{b}\|$ be small, and let $\vec{x}' = A^{-1}\vec{b}'$, $\vec{x} = A^{-1}\vec{b}$. Then we have that

$$\|\vec{x}' - \vec{x}\| = \|A^{-1}\vec{b}' - A^{-1}\vec{b}\| \leq \|A^{-1}\| \cdot \|\vec{b}' - \vec{b}\|, \tag{39}$$

where $\|A^{-1}\|$ denotes the induced matrix norm, and in this case the absolute condition number of A at $\vec{b}$. However, recall that the absolute condition number tells us nothing about the number of correct digits in the answer, so we need the relative condition number, which can be computed as

$$\frac{\|\vec{x}' - \vec{x}\|}{\|\vec{x}\|} \leq \|A^{-1}\| \cdot \frac{\|\vec{b}' - \vec{b}\|}{\|\vec{x}\|} = \|A^{-1}\| \cdot \frac{\|\vec{b}' - \vec{b}\|}{\|\vec{b}\|} \cdot \frac{\|\vec{b}\|}{\|\vec{x}\|} = \|A^{-1}\| \cdot \frac{\|\vec{b}' - \vec{b}\|}{\|\vec{b}\|} \cdot \frac{\|A\vec{x}\|}{\|\vec{x}\|} \leq \|A^{-1}\|\|A\| \cdot \frac{\|\vec{b}' - \vec{b}\|}{\|\vec{b}\|}, \tag{40}$$

so that $\|A^{-1}\|\|A\|$ is the relative condition number of A at $\vec{b}$.

The Singular Value Decomposition

If $A \in \mathbb{R}^{m\times n}$ with $m \geq n$, then we can write

$$A = U\Sigma V^\top, \tag{41}$$

where $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ are diagonal, and $\Sigma \in \mathbb{R}^{m\times n}$ is a rectangular diagonal matrix.

2/8 Lecture

Recall that the Taylor series of a scalar-valued function of several variables is

$$f = f(x_1, \cdots, x_n) = f(\vec{y}) + \sum_j \frac{\partial f}{\partial x_j}(x_j - y_j) + \textit{higher order terms.} \tag{42}$$

3.1 Multivariable Vector-Valued Taylor Series

Consider the multivariable vector-valued function

$$\vec{f}(\vec{x}) = \begin{pmatrix} f_1(x_1, \cdots, x_m) \\ \vdots \\ f_n(x_1, \cdots, x_m) \end{pmatrix}. \tag{43}$$

Place the expansions back into $\vec{f}$ and collect the terms, we have that

$$\begin{aligned} \vec{f}(\vec{x}) &= \begin{pmatrix} f_1(\vec{y}) + \sum_{k=1}^n \frac{\partial f_1(\vec{y})}{\partial x_k}(x_k - y_k) + \cdots \\ \vdots \\ f_n(\vec{y}) + \sum_{k=1}^n \frac{\partial f_n(\vec{y})}{\partial x_k}(x_k - y_k) + \cdots \end{pmatrix} = \vec{f}(\vec{y}) + \begin{pmatrix} \frac{\partial f_1(\vec{y})}{\partial x_1} & \cdots & \frac{\partial f_1(\vec{y})}{\partial x_m} \\ \vdots & & \vdots \\ \frac{\partial f_n(\vec{y})}{\partial x_1} & \cdots & \frac{\partial f_n(\vec{y})}{\partial x_m} \end{pmatrix} \begin{pmatrix} x_1 - y_1 \\ \vdots \\ x_m - y_m \end{pmatrix} \\ &= \vec{f}(\vec{y}) + J(\vec{y})(\vec{x} - \vec{y}) + (\vec{x} - \vec{y})^\top \cdot \frac{Q(\vec{y})}{2} \cdot (\vec{x} - \vec{y}) + \textit{higher order terms}, \end{aligned} \tag{44}$$

where J is called the *Jacobian matrix*, and Q is a *tensor* defined so that the ith entry of the second-order term $(\vec{x} - \vec{y})^\top \cdot \frac{Q(\vec{y})}{2} \cdot (\vec{x} - \vec{y})$ is given by

$$\frac{1}{2}\sum_{k,l} \frac{\partial^2 f_i(\vec{y})}{\partial x_k \partial x_l}(x_k - y_k)(x_l - y_l) = \frac{1}{2}(\vec{x} - \vec{y})^\top H_i(\vec{y})(\vec{x} - \vec{y}), \tag{45}$$

and H_i is the *Hessian* for f_i.

3.2 Multivariate Newton's Method

We want to solve a square system $\vec{f}(\vec{x}) = \vec{0}$ (*i.e.*, $m = n$). In general, determining whether there is one, none, or multiple solutions is exceedingly difficult. However, if a approximate $\vec{f}$ at $\vec{x}_0$ near the root $\vec{\xi}$, then we have

$$\vec{f}(\vec{x}) \approx \vec{f}(\vec{x}_0) + J(\vec{x}_0)(\vec{x} - \vec{x}_0), \tag{46}$$

also known as the truncated multivariable Taylor series. Evaluting at $\vec{xi}$, we have that

$$\vec{f}(\vec{x}_i) = \vec{0} \approx \vec{f}(\vec{x}_0) + J(\vec{x}_0)(\vec{\xi} - \vec{x}_0) \implies -\vec{f}(\vec{x}_0) \approx J(\vec{x}_0)(\vec{\xi} - \vec{x}_0) \implies \vec{\xi} \approx \vec{x}_0 - J^{-1}(\vec{x}_0)\vec{f}(\vec{x}_0). \tag{47}$$

Therefore, the Newton iteration is

$$\vec{x}_{k+1} = \vec{x}_k - J^{-1}(\vec{x}_k)\vec{f}(\vec{x}_k). \tag{48}$$

It can be shown that convergence is also quadratic, meaning that

$$\|\vec{x}_{k+1} - \vec{\xi}\|_2 \approx \|\vec{x}_k - \vec{\xi}\|_2^2. \tag{49}$$

Much like Newton's method, quadratic convergence may suffer if we have a singular Jacobian $J(\vec{x}_k)$. Another commonly seen form of the multivariate Newton's method is

$$J(\vec{x}_k)(\vec{x}_{k+1} - \vec{x}_k) = f(\vec{x}_k), \tag{50}$$

which can be done by solving $J(\vec{x}_k)\vec{s}_k = f(\vec{x}_k)$ and setting $\vec{x}_{k+1} = \vec{x}_k + \vec{s}_k$ in the iterative step. We will revise this when we talk about optimization.

2/13 Lecture

4 Optimization

4.1 Single Variable Optimization

Recall that root finding solves $f(x) = 0$. If we instead want to maximize or minimize f, we want to look for x such that $f'(x) = 0$. Therefore, we can apply our root finding methods to the equation $f'(x) = 0$ instead. Newton's method then becomes

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}. \tag{51}$$

However, note that for a general continuous function, $f'(x) = 0$ does not necessarily imply a global optimum. It can imply local optimums, or even non-optimal points (for instance, $x = 0$ for $f(x) = x^3$). One (common) regime where we can guarantee a minimum or maximum is when f is convex, defined as follows.

Definition 4.1. f is *convex* on $[a, b]$ if

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y), \tag{52}$$

for all $0 \leq \alpha \leq 1$ and $x, y \in [a, b]$.

Theorem 4.2. If f is convex on $[a, b]$, then any local minimum of f is also a global minimum of f.

Proof. We prove by contradiction. Let x^* be a local minimum of f, and $\tilde{x} \neq x^*$ be such that $f(\tilde{x}) < f(x^*)$. Then, since f is convex, we have that

$$f(\alpha x^* + (1 - \alpha)\tilde{x}) \leq \alpha f(x^*) + (1 - \alpha)f(\tilde{x}) < \alpha f(x^*) + (1 - \alpha)f(x^*) = f(x^*). \tag{53}$$

But since f is convex and x^* is a local minimum of f, there exists some small neighborhood $B_\epsilon(x^*)$ around x^*, such that $f(x) \geq f(x^*)$ for all $x \in B_\epsilon(x^*)$. Taking $\alpha = 1 - \frac{\epsilon}{2|x^* - \tilde{x}|}$ (ϵ can be arbitrarily small so we can guarantee that $0 \leq \alpha \leq 1$), we observe that

$$|\alpha x^* + (1 - \alpha)\tilde{x} - x^*| = (1 - \alpha)|x^* - \tilde{x}| = \frac{\epsilon}{2} < \epsilon, \tag{54}$$

so $\alpha x^* + (1 - \alpha)\tilde{x} \in B_\epsilon(x^*)$. But (53) then leads to a contradiction, so the proof is complete. $\square$

Theorem 4.3. If f is *strictly convex*, then there is only one local minimum.

Proof. Strictly convexity requires the inequality in (52) to be strict. The proof is trivial. □

4.2 Multivariate Optimization

Multivariate Newton's method. Now consider that we want to find

$$\vec{x}^* = \arg\min_{\vec{x}\in\mathscr{X}} f(\vec{x}) = \arg\min_{\vec{x}\in\mathscr{X}} f(x_1, x_2, \cdots, x_n). \tag{55}$$

Assume that f is smooth and strictly convex, then we can (hopefully) solve this using Newton's method on the system $\vec{\nabla} f = \vec{0}$, which is equivalent to $\partial f/\partial x_i = 0$ for all $i = 1, \cdots, n$. Then Newton's method says that

$$\begin{aligned}
\vec{x}_{k+1} &= \vec{x}_k - \begin{pmatrix} \frac{\partial}{\partial x_1}\left(\frac{\partial f(\vec{x}_k)}{\partial x_1}\right) & \cdots & \frac{\partial}{\partial x_n}\left(\frac{\partial f(\vec{x}_k)}{\partial x_1}\right) \\ \vdots & & \vdots \\ \frac{\partial}{\partial x_1}\left(\frac{\partial f(\vec{x}_k)}{\partial x_n}\right) & \cdots & \frac{\partial}{\partial x_n}\left(\frac{\partial f(\vec{x}_k)}{\partial x_n}\right) \end{pmatrix}^{-1} \vec{\nabla} f(\vec{x}_k) \\
&= \vec{x}_k - \begin{pmatrix} \frac{\partial^2 f(\vec{x}_k)}{\partial x_1 \partial x_1} & \cdots & \frac{\partial^2 f(\vec{x}_k)}{\partial x_1 \partial x_n} \\ \vdots & & \vdots \\ \frac{\partial^2 f(\vec{x}_k)}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f(\vec{x}_k)}{\partial x_n \partial x_n} \end{pmatrix}^{-1} \vec{\nabla} f(\vec{x}_k) = \vec{x}_k - H^{-1}(\vec{x}_k)\vec{\nabla} f(\vec{x}_k),
\end{aligned} \tag{56}$$

where H denotes the Hessian matrix. There is one computational challenge with this: evaluating and inverting the Hessian matrix are expensive, taking $O(n^2/2)$ and $O(n^3)$ time respectively. To avoid explicit construction of H, we consider *Quasi-Newton methods*, and recall that the secant method is the most basic Quasi-Newton method.

Broyden's Update. The goal is to form an approximation to H on each step of the multivariate Newton's method for optimization, and to be able to cheaply update this approximation. For instance, we want something like

$$\vec{x}_1 = \vec{x}_0 - H^{-1}(\vec{x}_0)\vec{\nabla} f(\vec{x}_0), \quad \vec{x}_2 = \vec{x}_1 - H^{-1}(\vec{x}_1)\vec{\nabla} f(\vec{x}_1), \quad H(\vec{x}_1) = H(\vec{x}_0) + \textit{something}. \tag{57}$$

Now consider the implied linear approximation from the multivariate Newton's method for optimization, that is,

$$\vec{\nabla} f(\vec{x}_{k+1}) \approx \vec{0} \approx \underbrace{\vec{\nabla} f(\vec{x}_k)}_{=:\vec{\nabla} f_k} + \underbrace{H(\vec{x}_k)}_{:=H_k}\underbrace{(\vec{x}_{k+1} - \vec{x}_k)}_{=:\vec{s}_k} \implies H_k \vec{s}_k \approx -\vec{\nabla} f_k. \tag{58}$$

To continue the iteration, we want to update the approximate Hessian matrix to H_{k+1}. Similar to the secant method, it should also be the case that

$$\vec{\nabla} f_{k+1} - \vec{\nabla} f_k \approx H_{k+1}(\vec{x}_{k+1} - \vec{x}_k) \implies H_{k+1}\vec{s}_k \approx \vec{\nabla} f_{k+1} - \vec{\nabla} f_k. \tag{59}$$

Of course this does not uniquely determine H_{k+1} given $\vec{s}_k$, $\vec{\nabla} f_{k+1}$, and $\vec{\nabla} f_k$ are known. We need an extra constraint that $H_{k+1} - H_k$ is of rank 1. This gives that

$$H_{k+1} = H_k + \frac{1}{\vec{s}_k^\top \vec{s}_k}\left(\vec{\nabla} f_{k+1} - \vec{\nabla} f_k - H_k \vec{s}_k\right)\vec{s}_k^\top. \tag{60}$$

Note that the form $\vec{u}\vec{v}^\top$ is the outer product $\vec{u} \otimes \vec{v}$, while the form $\vec{u}^\top \vec{v}$ is the inner product $\vec{u} \cdot \vec{v}$. Therefore, we propose the corresponding algorithm as follows. First, we initialize H_0 (finite differences, diagonal, etc.). Then, we set $\vec{x}_1 = \vec{x}_0 - H_0^{-1}\vec{\nabla} f(\vec{x}_0)$ and update to H_1 based on (60). Iterate this step until the terminating condition is met.

Remark 4.4. Since we are interested only in H^{-1}, there is actually a better way to update H_{k+1}^{-1} given H_k^{-1} without referring to H_{k+1} or H_k. Note that the updated Hessian matrix is in the form

$$H_{k+1} = H_k + \vec{u}\vec{v}^\top. \tag{61}$$

Therefore, we can use the *Sherman-Morrison-Woodbury formula*, which states that

$$(A + \vec{u}\vec{v}^\top)^{-1} = A^{-1} - \frac{A^{-1}\vec{u}\vec{v}^\top A^{-1}}{1 + \vec{v}^\top A^{-1}\vec{u}}. \tag{62}$$

2/15 Lecture

5 Numerical Linear Algebra

In infinite precision-exact arithmetic, solving linear systems comes down to a question of whether the system is consistent or not. On the computer, however, the situation is different due to the round-off error and finite precision calculations. We first make a few remarks on Julia. In Julia, the jth column of a matrix can be accessed by `A[:,j]`. A submatrix can be extracted as `A[i1:i2,j1:j2]`. To generate a matrix, one can do something like `A=rand(5,4)`. To compute the induced p-norm, one can use `opnorm(A,p)`. Note that `p=Inf` is also an option.

5.1 Gaussian Elimination

We all know how Gaussian elimination works. Consider a 2×2 matrix

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}. \tag{63}$$

Then $A\vec{x} = \vec{b}$ can be solved by eliminating a_{21}, such that

$$\left(\begin{array}{cc|c} a_{11} & a_{12} & b_1 \\ a_{21} & a_{22} & b_2 \end{array} \right) \sim \left(\begin{array}{cc|c} a_{11} & a_{12} & b_1 \\ 0 & a_{22} - \frac{a_{12}a_{21}}{a_{11}} & b_2 - \frac{a_{12}a_{21}}{a_{11}} \end{array} \right). \tag{64}$$

This implies that

$$x_2 = \frac{b_2 - \frac{a_{12}a_{21}}{a_{11}}}{a_{22} - \frac{a_{12}a_{21}}{a_{11}}}, \qquad x_1 = \frac{b_1 - a_{12}x_2}{a_{11}}. \tag{65}$$

This process is called the *backward substitution*, but it can easily fail. For instance, if one or some of the denominators evaluate to zero, then the process can no longer proceed. A successful algorithm would need to involve pivoting rows, and we will return to this later. One question to consider for this moment is how expensive is Gaussian elimination. To see this, we need to count the number of floating-point operations required to put A in echelon form. Consider the following algorithm:

for columns $j = 1, \cdots, n-1$ **do**
  for rows $i = j+1, \cdots, n$ **do**
    Compute $\texttt{val} = a_{ij}/a_{jj}$;
    Compute $\text{row}_i - \texttt{val} \cdot \text{row}_j$;
    Compute $b_i - \texttt{val} \cdot b_j$;
  end for
end for

Within each inner loop, the first computation takes 1 operation, the second computation takes $2(n-j)$ computations, and the third operation takes 2 flops. Therefore, the total number of floating-point operations can be computed as

$$\sum_{j=1}^{n-1} \sum_{i=j+1}^{n} (2(n-j)+3) = \sum_{j=1}^{n-1} (n-j)(2(n-j)+3) = \Theta(n^3). \tag{66}$$

5.2 LU Factorization

Another way to think about Gaussian elimination is the LU factorization. Each row operation corresponds to a matrix multiplication by an *elementary* lower triangular matrix on the left. For instance, consider the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix}. \tag{67}$$

We can perform some elementary row operations on A such that

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ -7 & 0 & 1 \end{pmatrix}}_{L_1} \underbrace{\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix}}_{A} = \begin{pmatrix} 1 & 2 & 3 \\ 0 & -3 & -6 \\ 0 & -6 & -21 \end{pmatrix}, \tag{68}$$

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix}}_{L_2} \underbrace{\begin{pmatrix} 1 & 2 & 3 \\ 0 & -3 & -6 \\ 0 & -6 & -21 \end{pmatrix}}_{L_1 A} = \underbrace{\begin{pmatrix} 1 & 2 & 3 \\ 0 & -3 & -6 \\ 0 & 0 & -9 \end{pmatrix}}_{U}. \tag{69}$$

Therefore, we can see that $L_2 L_1 A = U$, which implies $A = L_1^{-1} L_2^{-1} U$. Note that $L_1^{-1} L_2^{-1}$ is also lower triangular, then solving $A\vec{x} = \vec{b}$ is equivalent to solving $LU\vec{x} = \vec{b}$, where L is a lower triangular matrix and U is an upper triangular matrix. To do this, we only need to solve $L\vec{y} = \vec{b}$ by forward substitution, and then solve $U\vec{x} = \vec{y}$ using backward substitution. Each of these steps take $O(n^2)$ operations. However, the LU factorization step requires the same number of operations as the Gaussian elimination, which is $O(n^3)$, thus dominating the runtime.

Gaussian elimination and LU factorization failure. We have to notice that Gaussian elimination and LU factorization may fail if there is a zero or a small number in the pivot position. The solution is to interchange some rows, resulting in a factorization of the form $PA = LU$, where P is a permutation (row exchanging) matrix. For instance, let $\epsilon <$ MachEp, then consider

$$\begin{pmatrix} \epsilon & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}. \tag{70}$$

The exact solution to this linear system is

$$x_1 = 1 + \frac{\epsilon}{\epsilon - 1}, \qquad x_2 = 1 - \frac{\epsilon}{\epsilon + 1}, \tag{71}$$

and note that x_1 and x_2 are both approximately 1. Without pivoting, in floating point arithmetic, Gaussian elimination will give us

$$\left(\begin{array}{cc|c} \epsilon & 1 & 1 \\ 1 & 1 & 2 \end{array}\right) \sim \left(\begin{array}{cc|c} \epsilon & 1 & 1 \\ 0 & 1 - 1/\epsilon & 2 - 1/\epsilon \end{array}\right) \approx \left(\begin{array}{cc|c} \epsilon & 1 & 1 \\ 0 & -1/\epsilon & -1/\epsilon \end{array}\right), \tag{72}$$

which would further result in

$$x_2 = 1, \qquad x_1 = \frac{1 - 1}{\epsilon} = 0, \tag{73}$$

throwing the wrong answer.

2/22 Lecture

Before discussing pivoted LU factorization, there is one special case where it can be shown that pivoting is not necessary: $\vec{x}^\top A\vec{x} > 0$, *i.e.*, A is a *symmetric positive definite* matrix. Note that this fact is one of the earliest results in numerical analysis.

5.3 Cholesky Factorization

If A is a symmetric positive definite matrix, then we can write it as $A = U^\top U$, since then $A^\top = U^\top U$ as well. The algorithm is straightforward. Set

$$U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix}, \tag{74}$$

where $u_{jj} \neq 1$ for all j (in fact, this condition is not necessary). Then, we have that

$$U^\top U = \begin{pmatrix} u_{11} & 0 & \cdots & 0 \\ u_{12} & u_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ u_{1n} & u_{2n} & \cdots & u_{nn} \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix} = \begin{pmatrix} u_{11}^2 & u_{11}u_{12} & \cdots & u_{11}u_{1n} \\ u_{12}u_{11} & \sum_{i=1}^2 u_{i2}^2 & \cdots & \sum_{i=1}^2 u_{i2}u_{in} \\ \vdots & \vdots & \ddots & \vdots \\ u_{1n}u_{11} & \sum_{i=1}^2 u_{in}u_{i2} & \cdots & \sum_{i=1}^n u_{in}^2 \end{pmatrix}. \tag{75}$$

Therefore, if we set $A = U^\top U$, we can iteratively compute $u_{11} = \sqrt{a_{11}}$, $u_{12} = a_{12}/u_{11}$, and so on. The total cost would be $\Theta(n^3/3)$ for the Cholesky factorization algorithm.

5.4 Row Pivoting

When pivoting is necessary when doing an LU decomposition, we can effectively write

$$L_m P_{m-1} \cdots L_3 P_2 L_2 P_1 L_1 A = U, \tag{76}$$

where each L_j is a lower triangular matrix and each P_j is a permutation matrix. By the term "permutation matrix", we refer to some matrix P_0 that can for instance interchange two rows, such that

$$P_0 A = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} - & \vec{a}_1^\top & - \\ - & \vec{a}_2^\top & - \\ - & \vec{a}_3^\top & - \\ - & \vec{a}_4^\top & - \end{pmatrix} = \begin{pmatrix} - & \vec{a}_1^\top & - \\ - & \vec{a}_2^\top & - \\ - & \vec{a}_4^\top & - \\ - & \vec{a}_3^\top & - \end{pmatrix}. \tag{77}$$

The question is, how to obtain an equivalent form of the factorization $PA = LU$, where P is the permutation matrix resulting from all of the row swaps in the factorization (possibly reordering all rows), L is a lower triangular matrix, and U is an upper triangular matrix. Moreover, note that since each P_j involves only a single row interchange (to be distinguished from P), we have that $P_j^{-1} = P_j = P_j^\top$. As an example, suppose that $L_2 P_1 L_1 A = U$. We then take $L_2 = L_2'$ and $L_1 = P_1 L_1' P_1$. This thus implies that

$$L_2' P_1 P_1 L_1' P_1 A = U \implies L_2' L_1' P_1 A = U \implies P_1 A = (L_2' L_1')^{-1} U. \tag{78}$$

It can be easily shown that $L_1' = P_1 L_1 P_1$ and $L_2' = L_2$ are lower triangular, and therefore in the general case, all permutation matrices can be "moved" to the right by the above argument (details omitted, but make heavy use of the fact that $P_j = P_j^{-1}$ as above).

Gaussian elimination and LU factorization failure (revisited). Recall a previous example, in which we set $\epsilon <$ MachEp and consider

$$\begin{pmatrix} \epsilon & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}. \tag{79}$$

The exact solution to this linear system is

$$x_1 = 1 + \frac{\epsilon}{\epsilon - 1}, \qquad x_2 = 1 - \frac{\epsilon}{\epsilon + 1}, \tag{80}$$

but Gaussian elimination without pivoting would result in

$$x_2 = 1, \qquad x_1 = \frac{1-1}{\epsilon} = 0. \tag{81}$$

Now with row pivoting, we swap the first and second rows, so that

$$\left(\begin{array}{cc|c} 1 & 1 & 2 \\ \epsilon & 1 & 1 \end{array}\right) \sim \left(\begin{array}{cc|c} 1 & 1 & 2 \\ 0 & 1-\epsilon & 1-2\epsilon \end{array}\right) \approx \left(\begin{array}{cc|c} 1 & 1 & 2 \\ 0 & 1 & 1 \end{array}\right), \tag{82}$$

resulting in the correct answer

$$x_2 = 1, \qquad x_1 = \frac{2-1}{1} = 1. \tag{83}$$

5.5 Conditioning and Backward Stability

Recall from before that we computed the (relative) condition number of solving $A\vec{x} = \vec{b}$, which is

$$K(A) = \|A\|\|A^{-1}\|. \tag{84}$$

Theorem 5.1 (Perturbations of $A, \vec{x}$)**.** If $A(\vec{x} + \Delta\vec{x}) = \vec{b} + \Delta\vec{b}$, then

$$\frac{\|\Delta\vec{x}\|}{\|\vec{x}\|} \leq K(A)\frac{\|\Delta\vec{b}\|}{\|\vec{b}\|}. \tag{85}$$

If $(A + \Delta A)(\vec{x} + \Delta\vec{x}) = \vec{b}$, then

$$\frac{\|\Delta\vec{x}\|}{\|\vec{x}\|} \leq K(A)\frac{\|\Delta A\|}{\|A\|}, \qquad \text{as } \|\Delta A\| \to 0. \tag{86}$$

Note that for any induced matrix norm, we have that

$$1 = \|I\| = \|AA^{-1}\| \leq \|A\|\|A^{-1}\| = K(A), \tag{87}$$

so a condition number of 1 is the best we can hope for. In that case, the relative perturbation of the solution has the same size as that of the data. A condition number of size 10^t indicates that in floating point arithmetic, roughly t digits are lost (*i.e.*, become incorrect) in computing the solution $\vec{x}$. Moreover, if $K(A) > \textsc{MachEp}^{-1}$, then for computational purposes the matrix A is singular.

Residual and backward error. Suppose that $\tilde{x}$ is the computed solution to the exact linear system $A\vec{x} = \vec{b}$, then $\tilde{x} = \vec{x}$ is unknown since the exact solution $\vec{x}$ is unknown. Define the *residual* as

$$\vec{r} = \vec{b} - A\tilde{x}, \tag{88}$$

and note that $\tilde{x}$ solves the linear system $A\tilde{x} = \vec{b} - \vec{r}$, a perturbed linear system. The *backward error* is then $\|\vec{r}\|$, which the the perturbation from the original problem to the one that is solved exactly. However, note that small backward error does not necessarily imply small $\|\tilde{x} - \vec{x}\|$. To see this, let $\vec{h} = \tilde{x} - \vec{x}$ and thus $A\vec{h} = -\vec{r}$. By Theorem 5.1, we have that

$$\frac{\|\vec{x} - \tilde{x}\|}{\|\vec{x}\|} \leq K(A)\frac{\|r\|}{\|b\|}. \tag{89}$$

This says that the relative error can be much larger than the relative residual when the condition number $K(A)$ is large. In other words, when solving a linear system, all that can be expected is that the backward error be small, but we cannot guarantee a bound on the error.

5.6 QR Factorization

For the moment, let $A \in \mathbb{R}^{m\times n}$ with $m \geq n$ and $\text{rank}(A) = n$ (*i.e.*, n linearly independent columns). For a general $\vec{b} \in \mathbb{R}^m$, the system $A\vec{x} = \vec{b}$ has no solution if $m > n$. We can, however, ask for the least-squares solution

$$\vec{x}^* = \arg\min_{\vec{x}} \|A\vec{x} - \vec{b}\|_2. \tag{90}$$

In order to achieve this, we require that $\vec{x}^*$ satisfies the *normal equations*

$$A^\top(A\vec{x} - \vec{b}) = \vec{0} \implies A^\top A\vec{x} = A^\top\vec{b}. \tag{91}$$

To see this, let $\vec{y} \in \mathbb{R}^n$ be any vector, then

$$\begin{aligned}\|A(\vec{x} + \vec{y}) - \vec{b}\|_2^2 &= (A\vec{x} + A\vec{y} - \vec{b})^\top(A\vec{x} + A\vec{y} - \vec{b}) = (A\vec{x} - \vec{b})^\top(A\vec{x} - \vec{b}) + 2(A\vec{y})^\top(A\vec{x} - \vec{b}) + (A\vec{y})^\top(A\vec{y}) \\ &= \|A\vec{x} - \vec{b}\|_2^2 + 2y^\top A^\top(A\vec{x} - \vec{b}) + \|A\vec{y}\|_2^2 = \|A\vec{x} - \vec{b}\|_2^2 + \|A\vec{y}\|_2^2 \geq \|A\vec{x} - \vec{b}\|_2^2,\end{aligned} \tag{92}$$

which completes the proof. Furthermore, if we can write $A = QR$ with $Q \in \mathbb{R}^{m\times n}$ orthogonal (*i.e.*, $Q^\top Q = I$) and $R \in \mathbb{R}^{n\times n}$ upper triangular, the normal equations would become

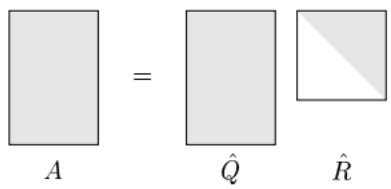
$$R^\top Q^\top Q R\vec{x} = R^\top Q^\top \vec{b} \implies R^\top R\vec{x} = R^\top Q^\top \vec{b}. \tag{93}$$

Clearly $R^\top$ is non-singular, so this is equivalent to solving $R\vec{x} = Q^\top\vec{b}$. This is an $n \times n$ square triangular linear system that can be easily solved by backward substitution, and has a solution $\vec{x} = R^{-1}Q^\top\vec{b}$, which is the desired least-squares solution.

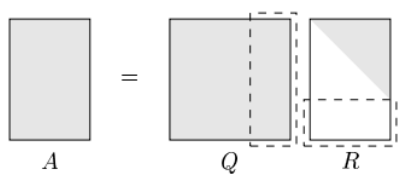
2/27 Lecture

There are in general two scenarios of QR factorization in the case $m \geq n$ and full rank n.

(I) Reduced QR factorization:

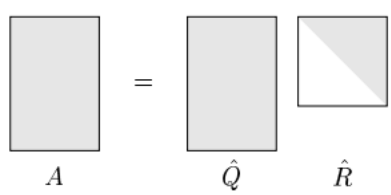


(II) Full QR factorization:



5.7 Gram-Schmidt (Triangular) Orthogonalization

In this section, consider the reduced QR factorization, *i.e.*, factorization of Type (I).



The simple (and naive) way to compute Q is via the *Gram-Schmidt process* which creates an orthonormal basis for the span of the columns of A, *i.e.*, $\vec{a}_1, \cdots, \vec{a}_n \in \mathbb{R}^m$. Write

$$A = \begin{pmatrix} | & | & & | \\ \vec{a}_1 & \vec{a}_2 & \cdots & \vec{a}_n \\ | & | & & | \end{pmatrix}, \qquad Q = \begin{pmatrix} | & | & & | \\ \vec{q}_1 & \vec{q}_2 & \cdots & \vec{q}_n \\ | & | & & | \end{pmatrix} \tag{94}$$

If $A = QR$, then since R is an upper triangular matrix, we have that

$$\begin{cases} \vec{a}_1 = & r_{11}\vec{q}_1, \\ \vec{a}_2 = & r_{12}\vec{q}_1 + r_{22}\vec{q}_2, \\ & \vdots \\ \vec{a}_n = & r_{1n}\vec{q}_1 + \cdots + r_{nn}\vec{q}_n. \end{cases} \tag{95}$$

The Gram-Schmidt process says: turn $\vec{a}_1, \cdots, \vec{a}_n$ into $\vec{q}_1, \cdots, \vec{q}_n$, so that Q can be a matrix with orthonormal columns (*i.e.*, an orthogonal matrix). The algorithm is then described as follows: on the jth step, set

$$\vec{v}_j = \vec{a}_j - (\vec{q}_1^\top \vec{a}_j)\vec{q}_1 - \cdots - (\vec{q}_{j-1}^\top \vec{a}_j)\vec{q}_{j-1}, \qquad \vec{q}_j = \frac{\vec{v}_j}{\|\vec{v}_j\|}. \tag{96}$$

Comparing with (95), this implies that

$$r_{ij} = \vec{q}_i^{\,\top} \vec{a}_j, \qquad |r_{jj}| = \left\| \vec{a}_j - \sum_{i=1}^{j-1} r_{ij}\vec{q}_i \right\|_2, \tag{97}$$

since we have that

$$r_{jj}\vec{q}_j = \vec{a}_j - r_{ij}\vec{q}_1 - \cdots - r_{j-1,j}\vec{q}_{j-1}. \tag{98}$$

The pseudocode of this classical Gram-Schmidt orthogonalization algorithm is shown as follows.

```
for j = 1 to n do
    v_j ← a_j;
    for i = 1 to j − 1 do
        r_ij ← q_i^T a_j;
        v_j ← v_j − r_ij q_i;     // round-off error can accumulate
    end for
    r_jj ← ‖v_j‖_2;
    q_j ← v_j / r_jj;
end for
```

Unfortunately, this algorithm is numerically unstable. The updating step in the first equation of (96) will require an iteration over $i = 1, \cdots, j-1$, potentially causing the the round-off error to accumulate. An alternative way to think about the Gram-Schmidt orthogonalization algorithm is as follows. Define

$$Q_{j-1} = \begin{pmatrix} | & | & & | \\ \vec{q}_1 & \vec{q}_2 & \cdots & \vec{q}_{j-1} \\ | & | & & | \end{pmatrix}, \tag{99}$$

and let P_j be the projector onto the subspace orthogonal to the columns of Q_{j-1}, that is,

$$P_j = I - Q_{j-1}Q_{j-1}^\top. \tag{100}$$

The classical Gram-Schmidt orthogonalization algorithm can then be written as

$$\vec{q}_1 = \frac{\vec{a}_1}{\|\vec{a}_1\|} = \frac{P_1\vec{a}_1}{\|P_1\vec{a}_1\|}, \qquad \cdots, \qquad \vec{q}_j = \frac{P_j\vec{a}_j}{\|P_j\vec{a}_j\|}. \tag{101}$$

This modified Gram-Schmidt algorithm is stable, and implicitly writes the projection in a different form

$$P_j = I - Q_{j-1}Q_{j-1}^\top = \underbrace{(I - \vec{q}_{j-1}\vec{q}_{j-1}^\top)}_{P_{\perp \vec{q}_{j-1}}}\underbrace{(I - \vec{q}_{j-2}\vec{q}_{j-2}^\top)}_{P_{\perp \vec{q}_{j-2}}}\cdots\underbrace{(I - \vec{q}_1\vec{q}_1^\top)}_{P_{\perp \vec{q}_1}}. \tag{102}$$

This is mathematically equivalent to (96), but numerically different. The pseudocode of this modified Gram-Schmidt orthogonalization algorithm is shown as follows.

```
for i = 1 to n do
    v_i ← a_i;     // Θ(m)
end for
for i = 1 to n do
    r_ii ← ‖v_i‖_2;     // m multiplications + m-1 additions + 1 square root
    q_i ← v_i / r_ii;     // m divisions
    for j = i + 1 to n do
        r_ij ← q_i^T v_j;     // m multiplications + m-1 additions
        v_j ← v_j − r_ij q_i;     // m multiplications + m subtractions
```

10: **end for**
11: **end for**

This modified Gram-Schmidt algorithm is the one used in practice, since it is less sensitive to the effects of rounding errors, and thus is a stable algorithm. The number of floating-point operations it need can be computed as

$$\sum_{i=1}^{n}\left(3m+\sum_{j=i+1}^{n}(4m-1)\right)\sim\sum_{i=1}^{n}\sum_{j=i+1}^{n}4m\sim\Theta(2mn^2). \tag{103}$$

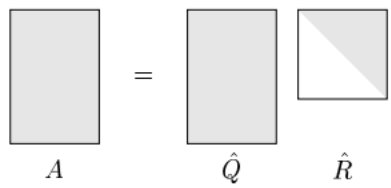
Therefore, this is a cubic algorithm (like Gaussian elimination, basically), and it is suited to reduced QR factorization, *i.e.*, factorization of Type (I).

5.8 Householder Reflection (Orthogonal) Triangularization

As we have seen previously, the Gram-Schmidt iteration applies a succession of elementary triangular matrices R_k on the right of A, so that the resulting matrix

$$A\underbrace{R_1R_2\cdots R_n}_{\hat{R}^{-1}}=\hat{Q} \tag{104}$$

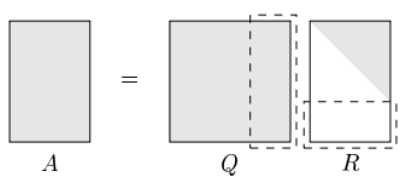
has orthonormal columns, *i.e.*, is an orthogonal matrix. The product $\hat{R}=R_n^{-1}\cdots R_2^{-1}R_1^{-1}$ is an upper triangular matrix, and thus $A=\hat{Q}\hat{R}$ is a reduced QR factorization of A, *i.e.*, a factorization of Type (I).



In contrast, the *Household method* that we are going to introduce applies a succession of elementary matrices Q_k on the left of A, so that the resulting matrix

$$\underbrace{Q_n\cdots Q_2Q_1}_{Q^\top}A=R \tag{105}$$

is an upper triangular matrix. The product $Q=Q_1^\top Q_2^\top\cdots Q_n^\top$ is orthogonal, and thus $A=QR$ would be a full QR factorization of A, *i.e.*, a factorization of Type (II).



As follows is an example of the Household method.

$$\begin{pmatrix}\times&\times&\times\\\times&\times&\times\\\times&\times&\times\\\times&\times&\times\\\times&\times&\times\end{pmatrix}\xrightarrow{Q_1}\begin{pmatrix}\star&\star&\star\\0&\star&\star\\0&\star&\star\\0&\star&\star\\0&\star&\star\end{pmatrix}\xrightarrow{Q_2}\begin{pmatrix}\times&\times&\times\\&\star&\star\\&0&\star\\&0&\star\\&0&\star\end{pmatrix}\xrightarrow{Q_3}\begin{pmatrix}\times&\times&\times\\&\times&\times\\&&\star\\&&0\\&&0\end{pmatrix}. \tag{106}$$

The question is, how can we construct orthogonal matrices Q_k, so as to introduce zeros as indicated above? The standard approach is as follows. Each Q_k is chosen to be a matrix of the form

$$Q_k=\begin{pmatrix}I_{k-1}&0\\0&F\end{pmatrix}, \tag{107}$$

where I_{k-1} is the $(k-1)\times(k-1)$ identity matrix and F is an $(m-k+1)\times(m-k+1)$ orthogonal matrix. Multiplication by F must introduce zeros in the kth column. The Householder algorithm chooses F to be a particular matrix called a *Household reflector*. Suppose that at the beginning of Step k, the entries $k, \cdots, m$ of the kth column are given by $\vec{x} \in \mathbb{R}^{m-k+1}$. To introduce the correct zeros into the kth column, the Household reflector F should effect the following map, such that

$$\vec{x} = \begin{pmatrix} \times \\ \times \\ \times \\ \vdots \\ \times \end{pmatrix} \quad \xrightarrow{F} \quad F\vec{x} = \begin{pmatrix} \|\vec{x}\| \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} = \|\vec{x}\|\vec{e}_1. \tag{108}$$

The idea is that, the reflector F should reflect the space $\mathbb{R}^{m-k+1}$ across the hyperplane H orthogonal to $\vec{v} = \|\vec{x}\|\vec{e}_1 - \vec{x}$, as is demonstrated in Figure 1.

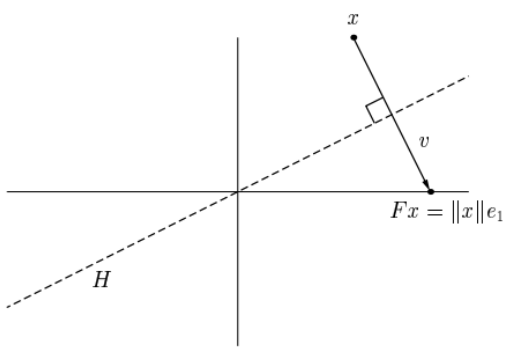


Figure 1: An illustration of a Household reflector.

Recall that for any $\vec{y} \in \mathbb{R}^p$, the *orthogonal projection* of $\vec{y}$ onto a hyperplane H that is orthogonal to some vector $\vec{v} \in \mathbb{R}^p$ can be written as

$$P\vec{y} = \left(I - \frac{\vec{v}\vec{v}^\top}{\vec{v}^\top\vec{v}}\right)\vec{y} = \vec{y} - \vec{v}\left(\frac{\vec{v}^\top\vec{y}}{\vec{v}^\top\vec{v}}\right). \tag{109}$$

Since a reflection would need to go twice the distance as a projection, the reflection would be written as

$$F\vec{y} = \vec{y} - 2\vec{v}\left(\frac{\vec{v}^\top\vec{y}}{\vec{v}^\top\vec{v}}\right) = \left(I - 2\frac{\vec{v}\vec{v}^\top}{\vec{v}^\top\vec{v}}\right)\vec{y}. \tag{110}$$

Hence, back to our case, the Householder reflector F and thus be selected as

$$F = I - 2\frac{\vec{v}\vec{v}^\top}{\vec{v}^\top\vec{v}}. \tag{111}$$

3/1 Lecture

Note that one could also reflect $\vec{x}$ to $-\|\vec{x}\|\vec{e}_1$ instead of reflecting to $\|\vec{x}\|\vec{e}_1$. However, we should always choose the one that moves $\vec{x}$ a longer distance for the reason of numerical stability. This can be simply done. For instance, if $x_1 \geq 0$ we can choose to reflect to $-\|\vec{x}\|\vec{e}_1$, and otherwise we can choose to reflect to $\|\vec{x}\|\vec{e}_1$. Now that we have

$$Q_n \cdots Q_2 Q_1 A = R, \tag{112}$$

then it suffices to construct Q by $Q = Q_1^\top Q_2^\top \cdots Q_n^\top = Q_1 Q_2 \cdots Q_n$ (recall that Q_k are orthogonal, and by construction symmetric).

5.9 The Singular Value Decomposition

Let $A \in \mathbb{R}^{m\times n}$ with $m \geq n$ and $\operatorname{rank}(A) = n$. Then A can be factorized as

$$A = USV^\top, \tag{113}$$

where $U \in \mathbb{R}^{m\times n}$ is orthogonal (*i.e.*, with orthonormal columns), $S \in \mathbb{R}^{n\times n}$ is diagonal with entries $\sigma_1 \geq \cdots \geq \sigma_n \geq 0$, and $V^\top \in \mathbb{R}^{n\times n}$ is orthogonal as well. We can think of U and $V^\top$ as rotation and reflection, and S as positive scaling. Mathematically, the singular value decomposition can be computed by observing that $A^\top A$ is positive semi-definite and therefore can be written as

$$A^\top A = VS^2V^\top, \tag{114}$$

according to *eigen-decomposition*. Then, by setting $U = AVS^{-1}$, we will see that $US = AV$, and thus $A = USV^\top$. However, this is numerically more unstable than dealing with A directly since

$$K(A^\top A) = K^2(A). \tag{115}$$

Instead, observe (assuming $m = n$) that the matrix

$$H = \begin{pmatrix} 0 & A^\top \\ A & 0 \end{pmatrix} \in \mathbb{R}^{2m\times 2m} \tag{116}$$

is orthogonal, and that if $A = USV^\top$, we will be able to deduce that

$$H \begin{pmatrix} V & V \\ U & -U \end{pmatrix} = \begin{pmatrix} V & V \\ U & -U \end{pmatrix} \begin{pmatrix} S & 0 \\ 0 & -S \end{pmatrix}. \tag{117}$$

This does not involve matrix squaring, but deals with a $2m \times 2m$ matrix instead of a $m \times n$ one. We will introduce more on how to efficiently compute singular value decompositions later.

Pseudoinverse

Recall that for $A \in \mathbb{R}^{m\times n}$ with $m \geq n$ and $\operatorname{rank}(A) = n$, the solution to the least-squares problem must satisfy the normal equations, that is,

$$A^\top A\vec{x} = A^\top \vec{b} \implies \vec{x} = (A^\top A)^{-1}A^\top \vec{b}. \tag{118}$$

The matrix $(A^\top A)^{-1}A^\top$ is known as the *pseudoinverse* of A, which is commonly denoted by A^+. Substituting A with its singular value decomposition, we have that

$$A^+ = (A^\top A)^{-1}A^\top = (VS^2V^\top)^{-1}(USV^\top)^\top = VS^{-1}S^{-1}V^{-1}VSU^\top = VS^{-1}U^\top. \tag{119}$$

In the case that $m > n$, clearly A is not invertible, but with the pseudoinverse, we have that

$$A^+A = VS^{-1}U^\top USV^\top = VS^{-1}SV^\top = VV^\top = I. \tag{120}$$

Last Modified: Mar 3, 2023.